

## Advanced Java and J2EE Lab Programs

Name : Tejeshwini M

Reg No : 24BTCE155

Unit 1:

1. Write a Java program to demonstrate the concept of classes and objects. Create a class named Student with data members such as student name and student age. Create an object of the class and display the student details using a method.

```
class Student { 2 usages
    String name; 2 usages
    int age; 2 usages

    Student(String name, int age) { 1 usage
        this.name = name;
        this.age = age;
    }

    void display() { 1 usage
        System.out.println("Student Name: " + name);
        System.out.println("Student Age: " + age);
    }
}

public class ass {
    public static void main(String[] args) {
        Student s = new Student( name: "Tejeshwini", age: 20);
        s.display();
    }
}
```

```
Student Name: Tejeshwini
Student Age: 20
```

2. Write a Java program to demonstrate the concepts of inheritance and polymorphism. Create a parent class named Animal and a child class named Dog. Use method overriding to achieve runtime polymorphism and display the behavior of both classes.

```
class Animal { 4 usages 1 inheritor
    void sound() { 2 usages 1 override
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal { 1 usage
    @Override 2 usages
    void sound() {
        System.out.println("Dog barks");
    }
}

R Rename usages
public class ass {
    public static void main(String[] args) {
        Animal a = new Animal();
        Animal d = new Dog();

        a.sound();
        d.sound();
    }
}
```

```
Animal makes a sound
Dog barks
```

Unit 2:

3. Write a Java Servlet program to demonstrate the use of doGet() and doPost() methods, session management, cookies, and servlet annotation. The program should accept user input through a form, store the username in a session and cookie, and display the details using a servlet.

## Index.html

```
<!DOCTYPE html>
<html>
<body>
<form action="UserServlet" method="post">
Username:
<input type="text" name="username">
<input type="submit" value="Submit">
</form>
</body>
</html>
```

## UserServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/UserServlet")
public class UserServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");

        HttpSession session = request.getSession();
        session.setAttribute("user", username);

        Cookie cookie = new Cookie("username", username);
        response.addCookie(cookie);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h2>Username: " + username + "</h2>");
    }

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.getWriter().println("GET Method Called");
    }
}
```

4. JSP, Request Dispatcher and JSTL Problem Statement Write a Java web application to demonstrate: • `setAttribute()` and `getAttribute()` •

forward() in Request Dispatcher • JSP implicit objects • Exception handling in JSP • JSTL tags The program should pass a message from a servlet to a JSP page and display it using JSTL.

### MessageServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/MessageServlet")
public class MessageServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        request.setAttribute("msg", "Welcome to JSTL Demo");

        RequestDispatcher rd =
            request.getRequestDispatcher("display.jsp");
        rd.forward(request, response);
    }
}
```

### 5. JDBC Database Connection and ResultSet Problem Statement

Write a Java program to demonstrate JDBC connectivity with MySQL database. The program should:

- Establish a database connection
- Create a statement object
- Execute a SQL query
- Retrieve and display records using ResultSet
- Handle exceptions

```

import java.sql.*;

public class JDBCExample {

    public static void main(String args[]) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/studentdb",
                "root",
                "password");

            Statement st = con.createStatement();

            ResultSet rs = st.executeQuery("SELECT * FROM student");

            while(rs.next()) {

                System.out.println(rs.getInt("id") + " "
                    + rs.getString("name"));

            }

            con.close();

        } catch (Exception e) {
            System.out.println(e);
        }

    }

}

```

## 6.JDBC Transaction Processing and Metadata Problem Statement

Write a Java program to demonstrate transaction processing and database metadata using JDBC with MySQL. The program should: •

- Connect to MySQL database
- Perform insert operations using transaction management
- Commit the transaction
- Display database metadata information
- Handle SQL exceptions

```

import java.sql.*;

public class TransactionDemo {

    public static void main(String args[]) {

        try {

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/studentdb",
                "root",
                "password");

            con.setAutoCommit(false);

            Statement st = con.createStatement();

            st.executeUpdate("INSERT INTO student VALUES(101,'Rahul')");

            con.commit();

            DatabaseMetaData db = con.getMetaData();

            System.out.println("Database: " + db.getDatabaseProductName());
            System.out.println("Version: " + db.getDatabaseProductVersion());

            con.close();

        } catch (Exception e) {

            System.out.println(e);

        }

    }

}

```

## Unit 4:

### 7. JDBC Transaction Processing and Metadata Problem Statement

Write a Java program to demonstrate transaction processing and database metadata using JDBC with MySQL. The program should: •

- Connect to MySQL database • Perform insert operations using transaction management • Commit the transaction • Display database metadata information • Handle SQL exceptions

```

import java.sql.*;

public class TransactionDemo {

    public static void main(String args[]) {

        try {

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/studentdb",
                "root",
                "password");

            con.setAutoCommit(false);

            Statement st = con.createStatement();

            st.executeUpdate("INSERT INTO student VALUES(101,'Rahul')");

            con.commit();

            DatabaseMetaData db = con.getMetaData();

            System.out.println("Database: " + db.getDatabaseProductName());
            System.out.println("Version: " + db.getDatabaseProductVersion());

            con.close();

        } catch (Exception e) {

            System.out.println(e);

        }

    }

}

```

8. Java Swing Calculator Application Problem Statement Write a Java Swing program to demonstrate the use of Swing components, containers, and controls by creating a simple calculator application. The program should:

- Use JFrame, JLabel, JTextField, and JButton
- Accept two numbers from the user
- Perform addition operation
- Display the result in the front-end application window

```
import javax.swing.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {

    JLabel l1,l2,l3;
    JTextField t1,t2,t3;
    JButton b;

    Calculator(){

        l1=new JLabel("Number 1");
        l2=new JLabel("Number 2");
        l3=new JLabel("Result");

        t1=new JTextField();
        t2=new JTextField();
        t3=new JTextField();

        b=new JButton("Add");

        l1.setBounds(20,20,100,20);
        t1.setBounds(120,20,100,20);

        l2.setBounds(20,60,100,20);
        t2.setBounds(120,60,100,20);

        l3.setBounds(20,100,100,20);
        t3.setBounds(120,100,100,20);

        b.setBounds(70,150,80,30);
```



```

        add(l1);
        add(l2);
        add(l3);

        add(t1);
        add(t2);
        add(t3);

        add(b);

        b.addActionListener(this);

        setSize(300,250);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e){

        int a=Integer.parseInt(t1.getText());
        int b=Integer.parseInt(t2.getText());

        int c=a+b;

        t3.setText(String.valueOf(c));
    }

    public static void main(String args[]){

        new Calculator();
    }
}

```

## 9. Spring Dependency Injection and Request Mapping Problem

Statement Write a simple Spring MVC application to demonstrate: • Inversion of Control (IoC) • Dependency Injection using Setter Injection • Spring Bean configuration • @RequestMapping • Spring MVC Controller The application should display a welcome message using Spring MVC.

WelcomeService.java

```

public class WelcomeService {

    public String getMessage() {
        return "Welcome to Spring MVC";
    }
}

```

WelcomeController.java

```

@Controller
public class WelcomeController {

    @Autowired
    WelcomeService service;

    @RequestMapping("/welcome")
    public String welcome(Model model){

        model.addAttribute("msg", service.getMessage());

        return "welcome";
    }
}

```

welcome.jsp

```

<html>
<body>

<h2>${msg}</h2>

</body>
</html>

```

## 10. Spring MVC Form Validation Application Problem Statement

Write a Spring MVC program to demonstrate:

- Spring MVC form handling
- Spring Validation
- Request Mapping
- Form Tag Library
- Dependency Injection

The application should accept student details through a form and validate whether the name field is empty. java programs

Student.java

```

import jakarta.validation.constraints.NotBlank;

public class Student {

    @NotBlank(message="Name cannot be empty")
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

## StudentController.java

```
@Controller
public class StudentController {

    @RequestMapping("/form")
    public String showForm(Model model){

        model.addAttribute("student",
            new Student());

        return "studentForm";
    }

    @RequestMapping("/save")
    public String save(

        @Valid Student student,
        BindingResult result){

        if(result.hasErrors()){

            return "studentForm";
        }

        return "success";
    }
}
```

## studentForm.jsp

```
<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>

<html>

<body>

<form:form action="save" modelAttribute="student">

Name:
<form:input path="name"/>

<form:errors path="name"/>

<input type="submit" value="Submit"/>

</form:form>

</body>

</html>
```